Ambiguity Localization at the Cost of a Forward Pass: Lightweight Hidden-State Probing for Clarification-Question Ranking in Agentic Code Generation

Author Name
Affiliation
Institution
City, Country
email@example.com

***Abstract*—Agentic coding assistants that ask a clarifying question before writing code can resolve specification ambiguity that a one-shot generation cannot. The standard way to decide *whether* and *what* to clarify is to estimate the coding model's uncertainty by repeated sampling of its own outputs – an approach that scales the cost of every clarification decision with the cost of the (often large) coding model itself. We show that a lightweight linear probe over the hidden states of a small, decoupled model (Qwen3-8B), extracted in a single forward pass, can (i) detect the degree of specification ambiguity in a coding prompt and (ii) rank candidate clarifying questions for a two-round elicit-rank-clarify-code agentic pipeline, at a fraction of the inference cost of prompting a model to reason about its own uncertainty. We evaluate the ranking probe against a zero-shot self-rank baseline across four benchmark sources (ClarifyCodeBench and three Orchid-derived variants of HumanEval and BigCodeBench) under grouped, leave-one-source-out splits, and against real, executable test suites in a live two-round clarify-then-code loop with two different coding models. We find the probe reaches ranking quality close to – and on its closest-to-training-distribution source, above – the zero-shot baseline, at substantially lower measured inference cost. Using the fractional test-case pass rate rather than an all-or-nothing correctness metric, we show that correctly ranking the labeled ambiguity to rank one is associated with a statistically significant round-1 correctness gain ($+9.9$ percentage points, $p = 0.035$, bootstrap 95% CI $[1.2, 19.1]$pp on Orchid-sourced problems), a result the coarser binary metric alone does not surface significantly. We further show that asking any clarifying question at all, independent of which one, is itself a significant and robust correctness gain, and that a cheap, targeted prompt augmentation recovers missing type information that the ambiguity-injection process otherwise strips, further improving pass rates. We report all findings, including places where our own earlier conclusions were revised under closer methodological scrutiny, as a case study in evaluating agentic clarification pipelines rigorously.**

***Index Terms*—agentic software engineering, code generation, ambiguity detection, hidden-state probing, clarifying questions, large language models**

I. Introduction

Large language model (LLM) coding assistants increasingly operate as *agents*: given a natural-language task description, they plan, write, run, and revise code with little further human input. A persistent failure mode in this setting is specification ambiguity – the prompt genuinely under-determines the intended behavior, and a one-shot generation commits to an arbitrary resolution that may not match the user's intent. A natural mitigation is to have the agent ask a clarifying question before writing code, turning a single generation step into a short elicit-then-generate loop [1]. This raises two sub-problems: *detecting* that a prompt is ambiguous enough to be worth interrupting for, and, given a set of candidate clarifying questions, *ranking* which one to ask.

Existing approaches to both sub-problems typically estimate the coding model's own uncertainty by sampling its outputs repeatedly and measuring disagreement among the samples – an approach with a long lineage in uncertainty quantification for LLMs, exemplified by semantic entropy [2] and its application to code-generation ambiguity in systems such as ClarifyGPT [1]. This is effective, but its cost scales linearly with the number of samples drawn *from the coding model itself*, which is often the largest, most capable, and most expensive model in the pipeline. Every clarification decision therefore costs as much as several full code-generation calls, before any code has actually been written.

We investigate a different design point, motivated by a simple observation: if a prompt's hidden-state representation already encodes how ambiguous it is, that signal can be read off in a single forward pass, without sampling any generations at all – and that forward pass can be run on a small model *decoupled* from whichever model eventually writes the code. This motivates the semantic-entropy-probe approach of Kossen et al. [3], who train a linear probe on a model's hidden states to predict semantic entropy (a property of the model's *output*

distribution) from a single forward pass, avoiding repeated sampling at inference time for that specific prediction task. We adapt this probing methodology to a different target relevant to agentic software engineering: not the entropy of a model's own generations, but the ambiguity of an *input* coding prompt, and, in a second stage, the relative quality of candidate clarifying questions for resolving that ambiguity.

Concretely, we make the following contributions:

- We show that a per-layer linear probe over Qwen3-8B's residual-stream activations, extracted from a single forward pass over an unmodified coding prompt, predicts the degree of injected specification ambiguity significantly above chance (Section V), and that this signal is present, though imperfectly transferable, across four differently-constructed ambiguity benchmarks.
- We build a ranking probe that scores candidate clarifying questions from batched forward passes over the prompt concatenated with each candidate, and deploy it in a live, two-round elicit-rank-clarify-code agentic loop with real executable test suites, comparing it against a zero-shot self-rank baseline (the coding model is simply asked to pick the best candidate) under grouped, leave-one-source-out evaluation.
- We report, honestly, that the probe's ranking quality is close to but does not uniformly exceed the zero-shot baseline – it wins on the benchmark closest to its training distribution and trails on others – and we frame our contribution accordingly as a *cost-quality tradeoff*: comparable ranking quality at substantially lower measured inference cost, since the probe requires no autoregressive generation from the coding model, rather than as an unconditional accuracy win.
- We investigate whether ranking quality (does the pipeline pick the labeled ambiguity as the top clarifying question) actually predicts downstream code correctness, and show that the answer depends on the correctness metric used: an all-or-nothing "did every test pass" metric shows no significant relationship, but the more informative fractional test-case pass rate reveals a real, statistically significant effect in round one of the clarification loop. We report both metrics and both outcomes, since this discrepancy is itself a methodological finding about how to evaluate agentic clarification systems.
- We identify and quantify a second, largely orthogonal lever for correctness: whether *any* clarifying question is asked at all is a robust, significant improvement over asking none, and a cheap, targeted prompt augmentation that restores type and requirement information the ambiguity-injection process incidentally strips further reduces trivial crash-class failures.

We deliberately report several points in this investigation where our own earlier conclusions did not survive closer scrutiny – most notably, an early result that the probe outperformed zero-shot ranking was later reversed under a larger, more carefully filtered evaluation. We treat this not as a weakness to hide but as an illustration of the methodological rigor agentic-system evaluations in software engineering research need: a result computed once, on one split, with one metric, is not yet a finding.

II. Related Work

A. Uncertainty Quantification via Sampling

Semantic entropy [2] estimates an LLM's uncertainty about a free-form generation by sampling multiple outputs, clustering them by semantic equivalence, and computing the entropy of the resulting cluster distribution. This requires K generation calls per uncertainty estimate and is the conceptual basis for a family of clarify-then-generate systems for code, including ClarifyGPT [1], which uses repeated sampling of the coding model's outputs to decide when an ambiguous requirement warrants a clarifying question. These approaches are effective but tie the cost of every clarification decision to K additional calls to the coding model itself, which is frequently the most expensive component of the pipeline.

B. Single-Forward-Pass Probing

Kossen et al. [3] introduce semantic entropy probes: a linear probe trained on a single forward pass' hidden states to predict the semantic entropy that would otherwise require K generations to estimate, amortizing the sampling cost of training many probes' worth of labels into a one-time cost, with cheap inference thereafter. Our probing methodology directly reuses this approach – per-layer linear probing of residual-stream activations, best-split binarization for layer selection, bootstrapped evaluation – but applies it to a different prediction target. Where Kossen et al. probe for a property of the model's own *output* distribution (how much its generations would disagree with each other), we probe for a property of the *input* prompt (how much unresolved specification ambiguity it contains) and, in a second stage, a property of a candidate answer relative to that input (how well it resolves the ambiguity, for ranking purposes). This is a substantively different prediction problem: no sampling, clustering, or generation from the model being probed is involved anywhere in our pipeline, at either training or inference time, and consequently the model being probed need not be the model doing the coding.

C. Ambiguity and Clarification in Software Engineering

Resolving requirements ambiguity before implementation is a long-standing concern in software engineering, predating LLM-based tooling. Recent LLM-era work targets this directly for code generation, constructing benchmarks with deliberately injected or naturally occurring ambiguity and evaluating whether models recognize the need to ask a clarifying question before writing code [4]. Our evaluation builds on ClarifyCodeBench and on Orchid-style ambiguity injections over HumanEval [5] and BigCodeBench [6], which construct matched clear/ambiguous prompt pairs by deleting or rewriting a targeted specification detail and pairing the injected prompt with tests that specifically exercise the resulting ambiguity.

D. Agentic Coding Benchmarks

Our live evaluation loop – eliciting candidate clarifying questions, ranking them, generating code conditioned on the top-ranked answer, and scoring the result against real executable test suites – follows the general pattern of agentic coding benchmarks that emphasize execution-based correctness over surface-level similarity to a reference solution [5], [6]. We report correctness using the fractional test-case pass rate rather than an all-or-nothing pass metric, and show in Section V that this choice materially affects which relationships in the data reach statistical significance – a point we believe is broadly relevant to how agentic coding pipelines should be evaluated, independent of our specific method.

III. Approach

A. Hidden-State Extraction

All probes read the residual stream of Qwen3-8B, run in bf16 precision with thinking mode explicitly disabled. For a given prompt, we apply the model's chat template, run a forward pass with `output_hidden_states=True` and no key-value caching, and extract the hidden state at the last token of the templated input (before any generation) at every one of the model's 37 residual-stream checkpoints (layer 0, the embedding output, through layer 36, the final block's output). No text is generated at any point in this extraction; the only computation performed is the forward pass itself.

B. Ambiguity-Degree Probe

The first probe answers a coarser question: given a coding prompt, how much unresolved specification ambiguity does it contain, expressed as an ordinal count $K \in \{1, 2, 3\}$ of independently ambiguous details? We extract the last-token hidden state of the raw, unmodified prompt at each layer and fit an independent multinomial logistic regression per layer on top of a 100-component PCA plus standardization pipeline, with the regularization strength C swept on a validation split. Layer and configuration selection use grouped cross-validation, splitting by source-problem identifier (`StratifiedGroupKFold`) so that no two rows derived from the same underlying problem appear on both sides of a split – a leakage guard that is load-bearing for this task, since near-duplicate prompts sharing a source problem would otherwise let a probe memorize surface features of a specific problem rather than learning a general ambiguity signal. We report ordinal correlation (Spearman's ρ between the predicted expected value $E[K]$ under the probe's class-probability output and the true K) as the primary metric, alongside majority-class, random, and prompt-length-only baselines computed on the identical split.

C. Clarifying-Question Ranking Probe

The second, and practically more important, probe addresses a different problem: given a pool of N candidate clarifying questions (with LLM-elicited answers) for an ambiguous prompt, which one should the agent actually ask? We treat this as pairwise ranking cast as classification, in the style of RankSVM/RankNet: each candidate's answer is concatenated onto the base prompt (`"{prompt}\n\nClarification: {answer}"`), and a batched forward pass over all N concatenated variants (one GPU call, N sequences) extracts N hidden states at a single selected layer. A classifier trained on tier-difference vectors between higher- and lower-quality candidates – where candidate quality tiers (0/1/2, unrelated/plausible/matches-the- labeled-ambiguity) are assigned by an independent LLM judge – scores each candidate, and the pipeline asks the top-scored one. We additionally evaluate a supervised-contrastive (SupCon) variant of this probe, which trains directly on raw (non-PCA-reduced) activations with a contrastive objective and, as reported in Section V, improves cross-source generalization at some cost to in-distribution accuracy relative to the PCA-based classifier.

Critically, this ranking probe's inference cost is $O(N)$ forward passes executed as a single batched GPU call, not the $O(1)$ single forward pass of the ambiguity-degree probe in Section III-B – each candidate's answer text must be present in the input for its hidden state to reflect that candidate specifically. The relevant cost comparison is therefore not "one forward pass vs. one generation call" but "one batched forward pass over N short sequences vs. one generation call that reasons over all N candidates inline," which we measure directly in Section V-C.

D. The Two-Round Elicit-Rank-Clarify-Code Loop

We deploy the ranking probe in an agentic pipeline that mirrors how a clarify-then-code assistant would operate in practice:

1) **Elicit**: given an ambiguous prompt, generate a pool of candidate clarifying questions and, for each, an oracle answer (an independent LLM answers the question as if it were the ground-truth requirement author).
2) **Rank**: score all candidates with the trained probe (or, for the zero-shot baseline in Section IV, ask the coding model to pick directly from a single inlined list) and select the top-ranked candidate.
3) **Clarify**: append the selected candidate's question-answer pair to the prompt.
4) **Code**: generate code conditioned on the clarified prompt, and run the real, executable test suite associated with the problem.

This constitutes round one. Round two repeats steps 1–4 once more, conditioned on the round-one clarified prompt, to model an agent that gets a second opportunity to resolve any remaining ambiguity before finalizing its code. We report both round-1 and round-1+2 correctness throughout Section V, since, as we show, the value of a second clarification round does not straightforwardly track the value of the first.

E. Design Rationale for the Probe-Host Model

We use Qwen3-8B as the probe-host model throughout due to compute constraints in this study; the ranking probe's forward-pass cost scales with this model's size, not with the size of whichever model ultimately writes the code. This decoupling is itself part of the approach's intended

value proposition: ambiguity localization can, in principle, be performed by a small, cheap model regardless of which (possibly much larger and more expensive) model is used for code generation, whereas a self-consistency baseline built on the coding model's own repeated sampling ties uncertainty estimation cost directly to that larger model's per-token cost. We evaluate this decoupling directly in Section V by pairing the same ranking decisions with two different coding models of different capability.

IV. Experimental Setup

A. Datasets

We evaluate on four sources of ambiguity-injected coding problems:

- **ClarifyCodeBench** [4]: 419 problems derived from LiveCodeBench, each annotated with $K \in \{1, 2, 3\}$ deliberately deleted specification details.
- **Orchid-HEval**: HumanEval [5] problems rewritten with a single injected ambiguous edit (lexical, vagueness, semantic, or syntactic), paired with the original hidden tests.
- **Orchid-BCB** and **Orchid-BCB-Expand**: BigCodeBench [6] problems rewritten the same way, with BCB-Expand a larger, semi-automatically constructed extension of the original BCB subset.

All source-problem groupings are preserved under splitting: no two prompt variants derived from the same underlying problem appear on opposite sides of a train/validation/test or leave-one-source-out (LODO) split. We use grouped, stratified splits throughout the ambiguity-degree probe evaluation, and full leave-one-source-out evaluation (train on three sources, test on the fourth, held out entirely) for the ranking probe, to measure whether ranking quality generalizes across benchmark construction methodologies rather than only within one.

B. Models

The probe-host model is Qwen3-8B (bf16, thinking mode disabled) throughout. For code generation, we use two models of different capability: Qwen3-8B itself, and DeepSeek V3.2 (accessed via Bedrock), to test whether findings about clarification quality are specific to a particular coding model or hold across models of different capability. Candidate clarifying-question elicitation and independent quality judging (assigning tiers 0/1/2 to candidates relative to the benchmark's labeled ambiguity) use DeepSeek V3.2 as an independent judge, separate from whichever model generates the code under test.

C. Baselines

Zero-shot self-rank: the coding model is prompted with the ambiguous prompt and the full numbered list of candidate clarifying questions in a single message, and asked to output the number of the one it would ask. This requires exactly one generation call per problem (greedy decoding, no sampling), with the entire candidate pool inlined – not one call per candidate. **No-clarification**: the raw ambiguous prompt is passed directly to the coding model with no clarification step at all, establishing the baseline value of clarifying at all, independent of which question is chosen.

D. Correctness Metric

We report correctness as the **fractional test-case pass rate**: the number of individual test assertions a generated solution passes divided by the total number of test assertions for that problem, averaged across groups. This is distinct from – and, as shown in Section V, statistically more sensitive than – an all-or-nothing metric that scores a group as passing only if every test assertion succeeds. Both metrics are computable from the same underlying per-group pass/total counts; we report the fractional metric as primary throughout and note where it diverges from the all-or-nothing metric's conclusions.

E. Significance Testing

For between-groups comparisons (e.g., groups where the probe ranked the labeled ambiguity first vs. groups where it did not), we report Mann-Whitney U and Welch's t-test alongside a 10,000-resample bootstrap 95% confidence interval on the mean difference. For within-groups, paired comparisons (e.g., the same 244 problems with vs. without any clarification), we report the Wilcoxon signed-rank test, a paired t-test, and a paired bootstrap 95% CI on the mean of per-group differences. We disclose cases where these tests disagree at the margin rather than reporting only the test that reaches $p < 0.05$.

F. Compute Environment

All hidden-state extraction and generation runs on a single GPU (NVIDIA A10G, 23 GB, native bf16, no CPU weight offload). We note that an earlier measurement on a memory-constrained T4 GPU requiring CPU offload showed no measurable speedup between the probe and zero-shot generation, an artifact of a fixed CPU↔GPU weight-transfer cost dominating both paths equally; we report only native-GPU timings here, and describe the exact benchmarked comparison in Section V-C.

V. Results

A. Ambiguity-Degree Probe: A Single Forward Pass Encodes Some Signal

On ClarifyCodeBench restricted to $K \in \{1, 2, 3\}$ (excluding any constructed $K{=}0$ class, since every tested construction of a "no-ambiguity" class introduced a detectable confound – see Section VI), the ambiguity-degree probe achieves test Spearman $\rho = 0.312$ (5-fold cross-validated $\rho = 0.21 \pm 0.05$) at its best layer (layer 32 of 37), against a random baseline of $\rho = 0.007$ and a prompt-length-only baseline of $\rho = 0.043$. The layer-by-layer curve rises smoothly from $\rho \approx 0.08$ at layer 1 to its layer-32 peak, consistent with a property that is progressively constructed through the network's depth rather than a surface feature available immediately from the embedding layer. This result is modest but reproducible: an independent re-run with a differently-cleaned prompt template reproduced $\rho = 0.31230908$ to eight decimal places.

Extending this probe to a pooled, four-source binary (clear-vs-ambiguous) task spanning ClarifyCodeBench and all three Orchid sources, a PCA-reduced logistic-regression probe reaches pooled in-distribution AUROC of 0.75–0.77; the same probe's leave-one-source-out AUROC falls to 0.52–0.64, indicating the signal generalizes only partially across benchmark construction methodologies. A supervised-contrastive variant trained on raw (non-PCA) activations improves leave-one-source-out AUROC to 0.60 ± 0.01 (5 seeds) at some cost to pooled accuracy – a genuine generalization/accuracy tradeoff we revisit for the ranking probe below.

B. Ranking Probe vs. Zero-Shot: A Cost-Quality Tradeoff, Not an Accuracy Win

Table I reports rank-1 hit rate (did the pipeline's top choice match the benchmark's labeled ambiguity) for the trained ranking probe (PCA + logistic regression, the best-performing configuration) and for zero-shot self-rank, both evaluated on the identical 1,229-group, four-source pool under leave-one-source-out splitting.

TABLE I
RANKING PROBE VS. ZERO-SHOT SELF-RANK, RANK-1 HIT RATE, LEAVE-ONE-SOURCE-OUT EVALUATION (1,229 GROUPS).

Evaluation	Probe (PCA+LR)	Zero-shot Qwen
LODO mean (4 folds)	0.637	**0.710**
held-out ClarifyCodeBench (n=187)	**0.701**	0.663
held-out Orchid-HEval (n=187)	0.738	0.743
held-out Orchid-BCB (n=110)	0.582	**0.691**
held-out Orchid-BCB-Expand (n=745)	0.525	**0.742**
Pooled, proportional split (n=246)	0.581	**0.679**
Pooled, equal-per-source (n=94)	0.638	**0.713**

The trained probe does not uniformly beat zero-shot self-rank: zero-shot wins on 6 of the 7 rows in Table I, most substantially on Orchid-BCB-Expand (0.742 vs. 0.525, a 22-point gap). The probe's one clear win is on ClarifyCodeBench, its closest-to-training-distribution source. We note this result explicitly reverses an earlier, smaller-scale finding in our own development process, where the probe appeared to beat zero-shot by 13 points on a looser, 121-group evaluation split; that earlier result did not survive evaluation on the larger, more stringently filtered pool used here (Section VI).

We therefore frame the ranking probe's contribution as a **cost-quality tradeoff** rather than an accuracy win: it reaches ranking quality within a few points of zero-shot self-rank on two of four sources and its own training source, and trails meaningfully on the two BigCodeBench-derived sources, whose prompts are long and free-form relative to the size of the injected ambiguous edit (Section VI).

C. Inference Cost

On native GPU hardware (no CPU weight offload), a single probe forward pass measures 0.079s mean / 0.065s median per problem, against 0.38s mean / 0.37s median for zero-shot generation with the full candidate list inlined (n=20) – a $4.8\times$ speedup. **[Placeholder pending a fresh benchmark of the actual batched N-candidate extraction the deployed ranking pipeline performs, per Section III-C; the measurement above compares zero-shot generation to a single bare-prompt forward pass, which is not quite the right comparison and should be treated as provisional.]** We note that this measured advantage vanished entirely on a memory-constrained GPU requiring CPU weight offload (Section IV-F), underscoring that the compute-efficiency argument for this class of method depends on deployment hardware, not only on the underlying FLOP count.

D. Does Ranking Quality Predict Downstream Correctness?

We evaluate the live two-round elicit-rank-clarify-code loop (Section III-D) against real, executable test suites on 244 problems spanning all four sources, using DeepSeek V3.2 as the coding model. Table II reports the relationship between round-1 ranking success (`hit1`: did the top-ranked candidate match the labeled ambiguity) and round-1 fractional test-case pass rate, restricted to the three Orchid sources (ClarifyCodeBench lacks Orchid's one-injected-ambiguity, targeted-test design and is excluded from this specific comparison).

TABLE II
ROUND-1 FRACTIONAL PASS RATE BY RANKING OUTCOME, ORCHID SOURCES, DEEPSEEK V3.2 CODING MODEL (n=213).

Source	hit1=1	hit1=0	Gap
Orchid-BCB (n=21)	88.6%	67.2%	+21.5pp
Orchid-HEval (n=47)	100.0%	97.9%	+2.1pp
Orchid-BCB-Expand (n=145)	77.5%	65.7%	+11.8pp
All Orchid combined	**83.3%**	**73.3%**	+9.9**pp**

The all-Orchid-combined gap of $+9.9$ percentage points is statistically significant by Welch's t-test ($p = 0.035$) and by a 10,000-resample bootstrap 95% confidence interval on the mean difference, $[1.2, 19.1]$pp, which excludes zero. A Mann-Whitney U test on the same data is more conservative ($p = 0.10$); we report both rather than the more favorable test alone. This is a materially stronger result than the same relationship measured with the all-or-nothing pass metric on the same data ($+7.6$pp, Fisher's exact $p = 0.31$, not significant) – the fractional metric recovers a real effect that the coarser binary metric was not sensitive enough to detect at this sample size. We regard this discrepancy as a methodological finding in its own right: whether an agentic clarification pipeline appears to "work" can depend on which correctness metric an evaluation uses, independent of anything about the pipeline itself.

The effect is round-1-specific. Recomputed on round-1+2 (after a second clarification round), the all-Orchid-combined gap falls to $+7.4$pp ($p = 0.11$, bootstrap CI includes zero); per-source, Orchid-BCB alone strengthens to $+25.2$pp ($p = 0.05$, bootstrap CI $[5.7, 47.9]$pp, significant) while Orchid-BCB-Expand weakens to $+8.0$pp (not significant). We do not have a single clean explanation for this round-to-round variability at the current sample size and report it as an open question rather than force a narrative onto it.

We separately identify why the effect is markedly weaker for Orchid-BCB-Expand than for the other sources: an independent LLM-judged analysis of 1,952 candidate clarifying questions for this source finds that, while the officially labeled ambiguity is functionally relevant to correctness at essentially the same rate as every other source (91.3%, vs. 91.8–92.4% elsewhere), it is rarely the *only* relevant detail: on average 3.09 other candidate clarifications are independently functionally relevant to the same problem (vs. 1.48 for ClarifyCodeBench and 2.43 for Orchid-HEval), and the label is the sole relevant detail in only 7.0% of rows (vs. 26.7% and 14.1% respectively). Resolving the one labeled ambiguity correctly therefore caps, but does not determine, whether the generated code passes, because most Orchid-BCB-Expand problems are underspecified in several independent ways at once and the current pipeline resolves at most two (one per round).

E. Asking Any Question Beats Asking None

Independent of *which* clarifying question is asked, we find that asking any question at all is a robust, statistically significant improvement over asking none. Comparing the same 244 problems with vs. without any clarification (a matched, paired comparison), round-1 clarification improves the mean fractional pass rate by +4.0pp (Wilcoxon signed-rank $p = 0.0021$, paired bootstrap 95% CI $[1.6, 6.6]$pp), and round-1+2 by +6.3pp (Wilcoxon $p < 0.0001$, CI $[3.7, 9.1]$pp). Both results are significant under every test we applied, in contrast to the more marginal, metric-dependent ranking-quality effect in Section V-D. This suggests that, for this task family, the decision to clarify at all carries more of the achievable correctness gain than the precision of the ranking step that decides *which* clarification to make – a finding with direct design implications for agentic coding pipelines operating under a compute budget that cannot afford a highly precise ranking step.

F. A Cheap, Orthogonal Fix: Prompt Augmentation

Separately from the ranking pipeline, we identify and fix a source of trivial, crash-class failures unrelated to clarification quality: the Orchid-BCB and Orchid-BCB-Expand ambiguity injections rewrite BigCodeBench's free-text instruction prompt, which never carried parameter type information to begin with, causing the coding model to guess incorrect types for under-specified parameters (e.g., assuming a millisecond-epoch integer timestamp was a `datetime` object). Reinstating bare parameter types, requirements, and a worked example – without reinstating the full parameter-description text that would re-leak the injected ambiguity – and holding the ranking/clarification pipeline completely fixed, on the 106 problems where the exact round-1/round-2 clarified prompt text could be reconstructed without a new GPU run: round-1 mean fractional pass rate rises from 0.796 to 0.863, with the zero-pass (complete-crash) rate falling from 2.8% to 0.0%; round-2 rises from 0.804 to 0.854, zero-pass from 3.8% to 0.9%. This improvement is concentrated almost entirely in Orchid-BCB-Expand and is orthogonal to the ranking-quality question in Section V-D: it changes prompt content only, with the identical clarifying question selected in both conditions.

VI. Discussion and Threats to Validity

A. A Reversed Finding, Disclosed

An earlier stage of this investigation, evaluated on a smaller (121-group), more loosely filtered split, found the trained ranking probe outperforming zero-shot self-rank by 13 points. That result did not survive evaluation on the larger, more stringently filtered 1,229-group pool reported in Section V-B, where zero-shot wins on 6 of 7 rows. We report both results and the reversal explicitly rather than reporting only the final number, since a reversal under closer scrutiny is itself informative: it indicates the earlier comparison was underpowered or measuring an artifact of the looser filter, and a reader relying on the earlier number alone would draw the wrong practical conclusion about whether to deploy the probe in place of a zero-shot baseline.

B. Sample Size and Statistical Power

Several of the correctness results in Section V-D are drawn from modest sample sizes – most notably Orchid-BCB at n=21 for the hit1-vs-pass-rate comparison. Its +21.5pp round-1 gap is suggestive (Mann-Whitney $p = 0.057$) but not conclusive, and a swing of one or two cases would move it substantially; we report the wide bootstrap confidence interval alongside the point estimate for this reason rather than treating a directionally-interesting result as a settled one. The headline all-Orchid-combined result (n=213) is comparatively better powered and reaches significance by two of three tests applied, but we regard replication at larger n as necessary before treating the specific magnitude of the effect as established.

C. Cross-Source Generalization

The ranking probe's leave-one-source-out performance varies substantially by held-out source (Table I), transferring worst to Orchid-BCB-Expand. We traced this to the injected ambiguous edit constituting a small fraction of a long, free-form prompt for BigCodeBench-derived sources (median edit fraction 2.1–2.3% of prompt words) compared to Orchid-HEval's shorter, more directly-edited prompts (median 7.1%) – a single pooled or last-token hidden-state vector has proportionally less of its representational capacity available to encode a small, diluted edit within a long prompt. This is consistent with zero-shot self-rank, which re-reads the full prompt text at inference time rather than compressing it into one fixed-size vector, not suffering the same degradation on exactly this source. We consider this an important limitation of single-vector hidden-state probing for long, free-form specifications, rather than a limitation specific to our probe architecture, and a natural target for future work (e.g., multi-token or attention-weighted pooling that preserves more positional information about where in a long prompt the disambiguating content lives).

D. Choice of Probe-Host Model

We use Qwen3-8B as the probe-host model throughout due to compute constraints available to this study, not because it is necessarily the best choice. A larger probe-host model might encode ambiguity and candidate-quality information more richly, at the cost of increasing the per-forward-pass cost that is central to this approach's efficiency argument; conversely, a smaller model might preserve most of the cost advantage while sacrificing some signal quality. We do not have data characterizing this tradeoff and flag it as a direct avenue for future work rather than assume our specific model choice generalizes.

E. Metric Sensitivity

Section V-D's central methodological finding – that the fractional test-case pass rate recovers a significant ranking-quality effect that the all-or-nothing pass metric does not – depends on test suites that assign partial credit meaningfully, i.e., problems with multiple, at least partially independent test assertions. For problems with very few test cases, the two metrics converge and this distinction has little practical effect; we would expect the gap between metrics to shrink correspondingly on benchmarks with coarser test suites.

F. Absence of a Direct Repeated-Sampling Baseline

We compare against zero-shot self-rank, which requires a single generation call per problem, but we do not implement or benchmark a repeated-sampling, self-consistency-based baseline in the style of ClarifyGPT's underlying uncertainty-estimation mechanism [1] directly against our pipeline on the same problems. Any comparison to that class of method in this paper is therefore a related-work comparison grounded in its published methodology, not an empirical head-to-head measurement, and should be read accordingly; we regard implementing and directly benchmarking such a baseline as important future work to substantiate the cost-efficiency argument beyond the zero-shot-vs-probe comparison reported here.

G. Construct Validity of the Ambiguity Injections

Both ClarifyCodeBench and the Orchid-derived sources construct ambiguity by programmatically editing an originally unambiguous prompt, rather than sourcing naturally occurring ambiguous specifications. This is a common and practical approach to constructing matched clear/ambiguous pairs with known ground truth, but it means our results characterize how well this pipeline detects and resolves a specific, constructed style of ambiguity, and may not transfer identically to naturally occurring specification gaps in real development workflows.

VII. Conclusion and Future Work

We investigated whether a lightweight linear probe over a small, decoupled model's hidden states – extracted without sampling or generating any text – can detect specification ambiguity in coding prompts and rank candidate clarifying questions for an agentic clarify-then-code pipeline, as a lower-cost alternative to uncertainty estimation via repeated sampling of the coding model itself. Evaluated against a zero-shot self-rank baseline under grouped, leave-one-source-out splits across four benchmark sources, the probe reaches ranking quality close to, and on its training-distribution source above, the baseline, at substantially lower measured per-decision inference cost, framing this as a cost-quality tradeoff rather than an unconditional accuracy improvement. Deployed in a live, two-round elicit-rank-clarify-code loop against real executable test suites, we found that ranking the labeled ambiguity correctly is associated with a statistically significant round-1 correctness gain when measured by fractional test-case pass rate, though not when measured by an all-or-nothing pass metric on the same data – a discrepancy we treat as evidence that correctness-metric choice materially affects what an agentic clarification pipeline's evaluation can detect. We further found that asking any clarifying question at all, independent of ranking precision, is itself a robust and significant correctness improvement, and that a cheap, targeted prompt augmentation addressing a separate, orthogonal source of crash-class failures further improves outcomes.

We reported several points where earlier-stage conclusions in this investigation – most notably an initial finding that the probe outperformed zero-shot ranking – did not survive evaluation at larger scale and under stricter filtering, and disclose these reversals as part of the record rather than reporting only the final numbers. We believe this level of disclosure is necessary for agentic software engineering research broadly, where pipelines with several interacting stages (elicit, rank, clarify, generate, test) create many opportunities for an early, underpowered, or confounded result to look like a finding.

Future work includes: benchmarking directly against a repeated-sampling, self-consistency baseline on identical problems to substantiate the cost-efficiency argument empirically rather than by citation; measuring the actual batched N-candidate extraction cost against zero-shot generation, rather than the bare-prompt-only forward pass reported here; characterizing the tradeoff between probe-host model size and both ranking quality and inference cost; and improving cross-source generalization for long, free-form prompts where the ambiguous edit constitutes a small fraction of the total prompt, which we identified as the ranking probe's principal weakness.

References

[1] F. Mu, L. Shi, S. Wang, Z. Yu, B. Zhang, C. Wang, S. Liu, and Q. Wang, "Clarifygpt: Empowering llm-based code generation with intention clarification," in *arXiv preprint arXiv:2310.10996*, 2023.

[2] L. Kuhn, Y. Gal, and S. Farquhar, "Semantic uncertainty: Linguistic invariances for uncertainty estimation in natural language generation," *International Conference on Learning Representations (ICLR)*, 2023.

[3] J. Kossen, J. Han, M. Razzak, L. Schut, S. Malik, and Y. Gal, "Semantic entropy probes: Robust and cheap hallucination detection in LLMs," in *arXiv preprint arXiv:2406.15927*, 2024.

[4] "Clarifycodebench," 2024, placeholder citation – replace with the correct reference before submission.

[5] M. Chen, J. Tworek, H. Jun, Q. Yuan *et al.*, "Evaluating large language models trained on code," in *arXiv preprint arXiv:2107.03374*, 2021.

[6] T. Y. Zhuo, M. C. Vu, J. Chim *et al.*, "Bigcodebench: Benchmarking code generation with diverse function calls and complex instructions," in *arXiv preprint arXiv:2406.15877*, 2024.